ClauseClear AI

Intelligent Legal Contract Analysis Platform

Project Documentation Report

University: SRM University AP, Neerukonda 522240

Student Team:

M Bapu Koushik	AP23110010067
J Tharun	AP23110010644
V Kishore	AP23110010563

GitHub:

https://github.com/THARUN9959/Clause-Clear-AI

1 Project Overview

ClauseClear AI is a full-stack Generative AI web application designed for intelligent legal contract simplification and risk analysis. Powered by Google Gemini 2.5 Flash and built on Flask (Python), the platform enables users to upload or paste any legal contract and receive a structured, plain-language analysis in seconds.

The platform goes beyond simple text summarization by implementing six simultaneous context engineering principles in every AI call: a role-constrained system prompt, strict JSON schema input/output, session-based conversation memory (Memento Pattern), Retrieval-Augmented Generation (RAG) using keyword-based cosine similarity, benchmark-injected risk evaluation, and an autonomous self-critique agent loop that reviews and corrects its own output before returning results to the user.

A key differentiator is the **Indian Legal Playbooks** system — a selectable evaluation framework that grounds every risk assessment in specific Indian statutes including the Indian Contract Act (1872), the Digital Personal Data Protection Act (2023), and the Information Technology Act (2000). This ensures that all identified risks carry a formal legal citation rather than generic observations.

2 Problem Statement

Legal contracts are written in dense, technical language that is difficult for non-lawyers to understand. Most individuals and small businesses sign agreements without fully comprehending the obligations, risks, or clauses that may be unfavorable to them. Generic AI tools often hallucinate legal advice, apply inappropriate jurisdictional standards, or produce unstructured outputs that are not actionable.

There is a clear need for a context-engineered AI tool that can reliably analyze Indian-law contracts, cite specific statutes when flagging risks, produce machine-readable structured outputs, and maintain conversational context for follow-up questions — all without requiring the user to have any legal background.

3 Project Objectives

Technical Objectives

- Implement a multi-stage AI pipeline (classify → analyze → self-critique → embed → store) applying six context engineering principles.
- Build a RAG layer using keyword-based cosine similarity without external vector databases.
- Develop the Indian Legal Playbooks system with four distinct evaluation frameworks.

Performance Objectives

- Complete full unified analysis within 15–25 seconds.
- Return well-formed JSON on every call.
- Handle contracts up to 15,000 characters.

Deployment Objectives

- Run on localhost without cloud infrastructure beyond the Gemini API key.
- Single command startup (`python app.py`).
- Auto-create SQLite database on first launch.

Learning Outcomes

- Mastery of advanced Prompt Engineering techniques — system prompts, strict JSON schemas, few-shot prompting.
- Implementation of Retrieval-Augmented Generation (RAG) from scratch using cosine similarity mathematics.
- Development of full-stack REST APIs using Flask to mediate complex AI logic.
- Integration of persistent relational databases (SQLite) with session-based memory states.

4 Core Technologies Used

- **Python 3.10+** — Primary backend language for all application logic.
- **Flask 3.x** — Web framework: 13 routes handling upload, analysis, chat, history, and export.
- **Google Gemini 2.5 Flash** — Primary AI engine for contract analysis, classification, and self-critique.
- **DeepSeek (fallback #1), OpenAI (fallback #2), Anthropic Claude (fallback #3)** — Cascading AI provider fallback chain, tried in order when the primary provider is unavailable or rate-limited.
- **SQLite + SQLModel** — Persistent storage for analysis history, chat logs, and keyword embeddings.
- **FPDF2 / PyPDF2 / python-docx** — Server-side PDF generation and text extraction from uploaded PDF/DOCX files.
- **HTML5 / CSS3 / Vanilla JS** — Frontend: dark navy + amber theme, split layout, glassmorphism, animations.

5 Key Features

Full Contract Analysis (Unified Pipeline)

A single AI call returns a health score (0–100), health grade (A–F), risk breakdown (HIGH/MEDIUM/LOW), obligations tracker, key entities, and a plain-English verdict.

Self-Critique Loop

A second Gemini call reviews and corrects the primary analysis before it is returned to the user, significantly reducing hallucination risk.

RAG Context Injection

If a similar contract has been analyzed before, historical context is injected into the prompt to improve accuracy and consistency across repeated analyses.

Indian Legal Playbooks

The platform provides four selectable Indian legal frameworks:

- **General Commercial** (Indian Contract Act, 1872) — Standard commercial agreements.
- **Data Privacy & Tech** (DPDP Act 2023, IT Act 2000) — SaaS, data processing, tech contracts.
- **Pro-Vendor** (Indian Contract Act, 1872, Sec 74) — Freelancer and service provider reviews.
- **Pro-Buyer** (Indian Contract Act, 1872, Sec 27) — Procurement and vendor evaluation.

Every `HIGH_RISK` clause includes a `standard_justification` field citing the exact Indian statute.

Risk Severity Classification

- `HIGH_RISK` — Direct violation of Indian statute or severely one-sided clause (e.g. unlimited liability, no data breach notice). Action: immediate renegotiation.
- `MEDIUM_RISK` — Clause deviates from market standard (e.g. non-compete $>$ 12 months). Action: review and negotiate.
- `LOW_RISK` — Minor concern or missing best-practice clause (e.g. no force majeure). Action: note for awareness.

Benchmark-Injected Risk Evaluation

Market-standard ranges for 7 contract types are injected into prompts. The AI compares flagged clauses against these benchmarks to flag deviations objectively.

Session Memory

Last 10 conversation turns stored per Flask session and injected as `CONVERSATION MEMORY` into every Gemini call, enabling accurate follow-up question answers.

History and Export

SQLite-backed analysis history with pagination, server-side PDF report generation, and chat logs persisted alongside each analysis.

6 Target Users & Application Scenarios

The target users include Indian freelancers reviewing service agreements, startup founders evaluating SaaS and vendor contracts, legal studies students, small business owners, and non-technical professionals.

Scenario 1: Freelancer Reviews a Services Contract
A freelance developer receives a 12-page agreement. Using the Pro-Vendor playbook, the system flags an unlimited liability clause as `HIGH_RISK`, cites Section 74 of the Indian Contract Act, and proposes capping liability at six months of contract value.

Scenario 2: Startup Evaluates a Data Processing Agreement
A health-tech startup selects the Data Privacy & Tech playbook. The system flags the absence of a data breach notification obligation as `HIGH_RISK`, citing DPDP Act 2023 Section 8(6), and flags cross-border data transfer under Section 16.

Scenario 3: Employment Contract Review
An engineer uploads an offer letter. The General Commercial playbook flags a 24-month non-compete clause as `HIGH_RISK`, comparing it against the market standard of 6–12 months and citing Section 27 of the Indian Contract Act.

7 Technical Architecture

ClauseClear AI follows a layered architecture with clear separation between the presentation layer, application layer, AI pipeline layer, and persistence layer.

7.1 Layer Descriptions

Frontend Layer: A single-page split-layout interface built with Vanilla HTML5/CSS3/JavaScript. Features a dark navy (`#060A14`) background

with amber/gold (`#D4A853`) accents, glassmorphism card components, and animated loading stages.

Backend Layer (Flask): 13 discrete REST API routes handling file uploads, unified analysis, chat, session memory, and PDF export. Every route that triggers an AI call is rate-limited via `flask-limiter`.

AI Pipeline Layer: Enforces six context engineering principles on every AI generation request. See Figure 1.

Persistence Layer: A local SQLite database managing `AnalysisModel` (health scores, keyword embeddings, full JSON) and `ChatHistoryModel` (session-linked conversational turns). See Figure 2.

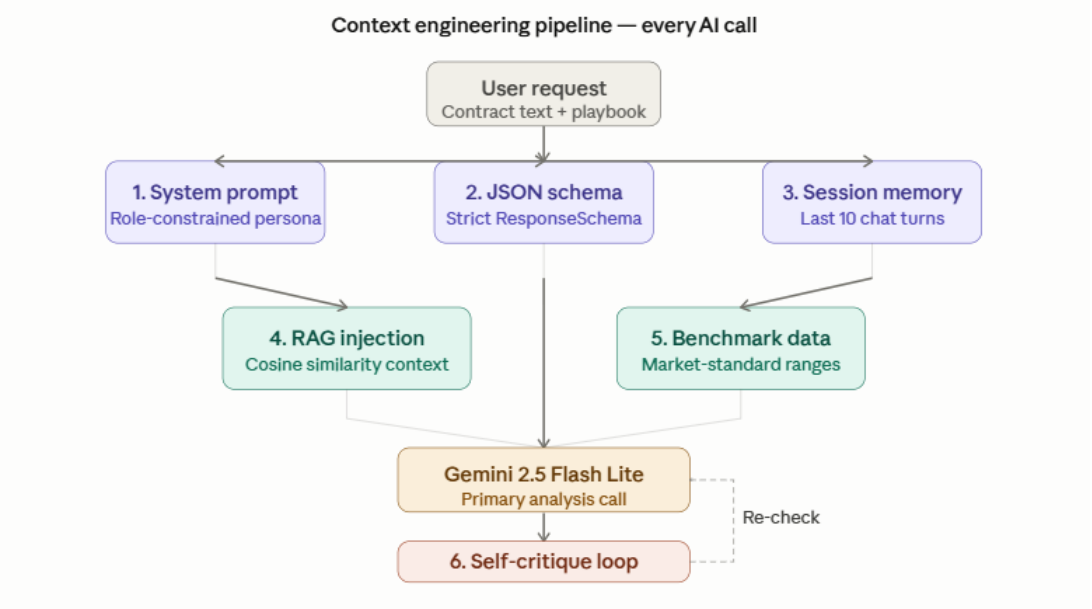

Figure 1: Context Engineering 6-Principles Pipeline

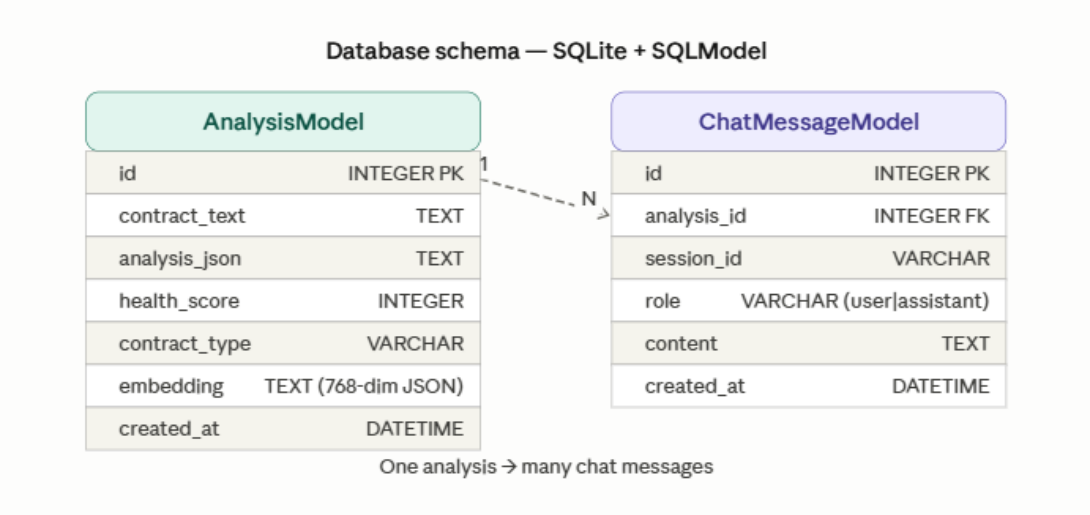

Figure 2: Database ER Diagram — AnalysisModel & ChatHistoryModel

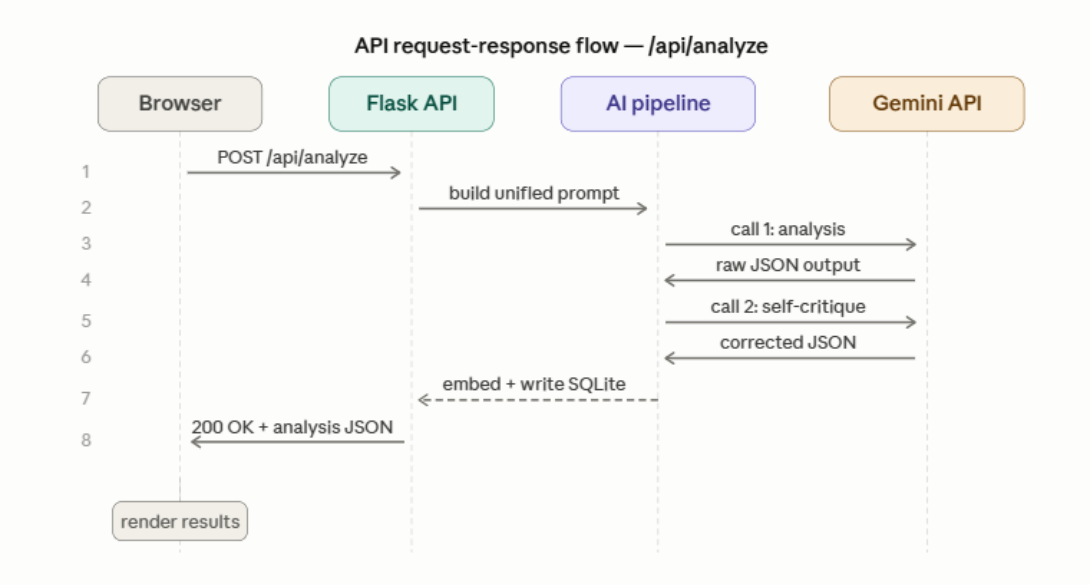

Figure 3: API Request-Response Flow (`/api/analyze`)

The `/api/analyze` endpoint orchestrates two sequential Gemini calls (primary analysis + self-critique) before writing the result to SQLite and returning validated JSON to the browser.

8 Pre-requisites & Prior Knowledge Required

Prior Knowledge Required

- **Python Fundamentals:** Functions, error handling, dictionaries, and asynchronous concepts.
- **Flask Web Framework:** Route decorators, request/response cycles, and session management.
- **REST API Principles:** GET, POST, DELETE methods, JSON payloads, and stateless architecture.
- **Relational Database Concepts:** Basic SQL, ORMs (SQLModel/SQLAlchemy), and schema design.
- **Generative AI & Prompt Engineering:** Context windows, system prompts, few-shot examples, and JSON schema enforcement.

Software & Hardware Requirements

- **Python:** 3.10 minimum, 3.11+ recommended
- **IDE:** VS Code or Cursor IDE
- **CPU:** Intel Core i5 / AMD Ryzen 5 (2 GHz+) minimum; i7/Ryzen 7 recommended
- **RAM:** 4 GB minimum, 8 GB recommended
- **OS:** Windows 10/11, macOS, or Ubuntu
- **Storage:** 500 MB free minimum, 2 GB recommended
- **Network:** Internet access required for Gemini API calls

Libraries and Dependencies

```
pip install flask google-genai sqlmodel fpdf2 PyPDF2 python-docx \
    python-dotenv anthropic flask-wtf flask-limiter
```

Listing 1: Installing all project dependencies

9 Project Workflow & Milestones

9.1 Milestone 1: Foundation and Document Ingestion

The first milestone establishes the entire project skeleton: the virtual environment, the Flask application scaffold, and the document extraction pipeline. At this stage, the application can accept a file upload, extract its raw text, and store it in session memory — ready for later AI processing.

Activity 1.1 — Environment Initialization

Create the project directory, set up the Python virtual environment, and create a `.env` file with the required API keys:

```
python -m venv venv
venv\Scripts\activate        # Windows
pip install flask google-genai sqlmodel fpdf2 PyPDF2 \
    python-docx python-dotenv anthropic flask-wtf flask-limiter
```

Listing 2: Environment setup and dependency installation

The `config.py` module loads all keys from `.env` and raises a `RuntimeError` if the mandatory `FLASK_SECRET_KEY` is missing, preventing silent misconfigurations:

```
class Config:
    GEMINI_API_KEY    = os.getenv("GEMINI_API_KEY",    "")
    DEEPSEEK_API_KEY  = os.getenv("DEEPSEEK_API_KEY",  "")
    OPENAI_API_KEY    = os.getenv("OPENAI_API_KEY",    "")
    ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY", "")

    SECRET_KEY = os.getenv("FLASK_SECRET_KEY")
    if not SECRET_KEY:
        raise RuntimeError(
            "FLASK_SECRET_KEY is required. "
            "Generate: python -c \"import secrets; print(secrets.
               token_hex(32))\""
        )
    GEMINI_MODEL  = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
    CLAUDE_MODEL  = os.getenv("CLAUDE_MODEL", "claude-3-5-haiku-latest")
```

Listing 3: config.py — API key loading and validation

Activity 1.2 — Flask Application Scaffold

The main `app.py` initializes Flask, optionally enables CSRF protection via `flask-wtf` and rate limiting via `flask-limiter`, then creates the SQLite

database on startup:

```
app = Flask(__name__)
app.config["SECRET_KEY"] = Config.SECRET_KEY
app.config["MAX_CONTENT_LENGTH"] = Config.MAX_CONTENT_LENGTH  # 10 MB

# Optional CSRF protection
try:
    from flask_wtf.csrf import CSRFProtect
    csrf = CSRFProtect(app)
except ImportError:
    csrf = None

# Optional rate limiting
try:
    from flask_limiter import Limiter
    from flask_limiter.util import get_remote_address
    limiter = Limiter(app=app, key_func=get_remote_address,
                      storage_uri="memory://")
except ImportError:
    limiter = None

# Create all SQLite tables on first run (idempotent)
create_db()
```

Listing 4: app.py — Flask initialization and startup sequence

A per-request SQLite session is opened via Flask's `g` object, ensuring thread-safe database access:

```
@app.before_request
def open_db():
    g.db = get_db_session()

@app.teardown_appcontext
def close_db(exc=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()
```

Listing 5: app.py — Per-request DB session management

Activity 1.3 — Document Extraction

The `/api/upload` endpoint accepts files up to 10 MB, runs MIME-type validation, extracts text using the appropriate parser, and stores the result in session memory:

```
@app.route("/api/upload", methods=["POST"])
def api_upload():
    file = request.files["file"]
    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type."}), 400

    filename = secure_filename(file.filename)
```

```
    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
    file.save(filepath)

    text = extract_text(filepath)   # dispatches to PDF/DOCX/TXT/OCR
        parser
    os.remove(filepath)             # delete after extraction

    mem = _get_memory()
    mem.set_contract(text, filename)

    return jsonify({
        "success": True,
        "filename": filename,
        "char_count": len(text),
        "preview": text[:800],
    })
```

Listing 6: app.py — /api/upload route

The `document_service.py` dispatches to the correct parser based on file extension. For PDFs with no text layer (scanned documents), it falls back to OCR via `pytesseract`:

```
def extract_text(filepath: str) -> str:
    ext = os.path.splitext(filepath)[1].lower()

    if ext == ".pdf":
        text = _extract_pdf_text(filepath)   # PyPDF2 text layer
        if not text.strip() and _OCR_AVAILABLE:
            text = _extract_pdf_ocr(filepath) # pytesseract fallback
        return text

    if ext == ".txt":
        return _extract_txt(filepath)

    if ext == ".docx":
        return _extract_docx(filepath)       # python-docx paragraphs +
            tables

    if ext in {".png", ".jpg", ".jpeg"}:
        return _extract_image_ocr(filepath)  # pytesseract direct

    return ""
```

Listing 7: document_service.py — Text extraction dispatcher

9.2 Milestone 2: Context Engineering Pipeline

This milestone implements the core intelligence of ClauseClear AI: the prompt assembly engine that injects all six context engineering principles simultaneously into every AI call.

Activity 2.1 — JSON Schema Enforcement

The AI is constrained to return structured JSON on every call. The unified prompt template includes an explicit JSON output schema so the response can be parsed directly without post-processing. All prompts use safe `.replace()` injection rather than `.format()` to prevent brace-collision errors from contract text that contains `{...}`:

```
prompt = (_UNIFIED_TEMPLATE
    .replace("SYSTEM_PROMPT_PLACEHOLDER",  BASE_SYSTEM_PROMPT)
    .replace("BENCHMARK_PLACEHOLDER",      benchmark_context + "\n" +
       type_addendum)
    .replace("PLAYBOOK_PLACEHOLDER",       playbook_text)
    .replace("HISTORICAL_PLACEHOLDER",     historical_context)   # RAG
    .replace("CONTRACT_TYPE_PLACEHOLDER",  contract_type)
    .replace("MEMORY_PLACEHOLDER",         memory_block)         #
       session memory
    .replace("CONTRACT_TEXT_PLACEHOLDER",  contract_text[:15000])
)
```

Listing 8: gemini_service.py — Safe prompt assembly with all 6 injections

Activity 2.2 — AI Provider Dispatcher

The `_call_ai()` function in `providers.py` implements a four-provider cascade. It tries Gemini first and falls back to DeepSeek, then OpenAI, then Claude, returning the first successful JSON response:

```
def _call_ai(prompt_text: str, temperature: float = 0.3) -> dict:
    # 1. Gemini (primary)
    result = _call_gemini(prompt_text, temperature)
    if "error" not in result:
        return result
    logger.warning("Gemini failed (%s) -- trying DeepSeek.", result.get(
       "error"))

    # 2. DeepSeek (first fallback)
    result = _call_deepseek(prompt_text, temperature)
    if "error" not in result:
        return result
    logger.warning("DeepSeek failed -- trying OpenAI.")

    # 3. OpenAI (second fallback)
    result = _call_openai(prompt_text, temperature)
    if "error" not in result:
        return result
    logger.warning("OpenAI failed -- trying Claude.")

    # 4. Claude (last resort)
    return _call_claude(prompt_text, temperature)
```

Listing 9: providers.py — Four-provider cascade dispatcher

Activity 2.3 — Benchmark Injection Service

Market-standard ranges for 7 contract types are hardcoded in `benchmark_service.py` and injected into every prompt. This grounds the AI's risk assessment against real-world norms rather than arbitrary judgment.

9.3 Milestone 3: Advanced AI Features & Playbooks

Activity 3.1 — Indian Legal Playbooks

Each playbook is a plain Python string stored in `playbooks.py`. The UI dropdown value is mapped to one of four strings and injected directly into the unified prompt via `PLAYBOOK_PLACEHOLDER`:

```
PLAYBOOKS = {
    "pro_vendor": """
    Evaluate this contract from the perspective of a vendor or service
    provider operating under Indian law.
    Flag unlimited liability clauses as HIGH_RISK (suggest capping at
    3-6 months of contract value per Indian market standard).
    Flag unilateral termination rights without a cure period as
       HIGH_RISK.
    Flag IP ownership clauses that assign all work product to the client
    without carving out pre-existing IP.
    Flag non-compete clauses broader than 12 months as HIGH_RISK.
    Evaluate under the Indian Contract Act, 1872.
    For every HIGH_RISK clause, provide a standard_justification
    from the vendor's perspective.
    """,

    "data_privacy_tech": """
    Apply the DPDP Act 2023 and IT Act 2000.
    Flag absence of data breach notification as HIGH_RISK
    under DPDP Act Section 8(6).
    Flag cross-border data transfer to non-notified countries
    as HIGH_RISK under DPDP Act Section 16.
    """,
}
DEFAULT_STANDARD = "general_commercial"
```

Listing 10: playbooks.py — Pro-Vendor playbook (excerpt)

The playbook is selected and injected in `gemini_service.py`:

```
playbook_text = PLAYBOOKS.get(evaluation_standard, PLAYBOOKS[
   DEFAULT_STANDARD])

prompt = (_UNIFIED_TEMPLATE
    .replace("PLAYBOOK_PLACEHOLDER", playbook_text)
)
```

Listing 11: gemini_service.py — Playbook selection and injection

Activity 3.2 — Self-Critique Agent Loop

After the primary analysis call returns, a second Gemini call is triggered. It receives the original contract text and the primary JSON output and is instructed to check for hallucinations, missing statute citations, and scoring inconsistencies. If the critiqued result is valid, it replaces the original:

```
if ENABLE_SELF_CRITIQUE:
    logger.info("Running self-critique | session=%s", session_id)
    critique_prompt = (SELF_CRITIQUE_PROMPT_TEMPLATE
        .replace("ORIGINAL_TEXT_PLACEHOLDER",
                 contract_text[:8000])
        .replace("ANALYSIS_JSON_PLACEHOLDER",
                 json.dumps(analysis, indent=2)[:6000])
    )
    critiqued = _call_ai(critique_prompt, temperature=0.2)
    if "error" not in critiqued and "health_score" in critiqued:
        analysis = critiqued   # replace with corrected output
        logger.info("Self-critique applied.")
    else:
        logger.warning("Self-critique unusable -- keeping original.")
```

Listing 12: gemini_service.py — Self-critique loop implementation

Activity 3.3 — RAG Implementation

The RAG layer uses a lightweight keyword-frequency vector instead of an external embedding API. For every analysis, a 300-dimensional TF-style vector is computed and stored. On the next analysis, cosine similarity is computed against all stored vectors; if the score exceeds 0.75, the matching historical analysis is injected as context:

```
def generate_embedding(text: str) -> list:
    words = re.findall(r'\b[a-z]{3,}\b', text.lower()[:8000])
    freq = {}
    for w in words:
        freq[w] = freq.get(w, 0) + 1
    total = sum(freq.values()) or 1
    top_words = sorted(freq.items(), key=lambda x: -x[1])[:300]
    vec = [v / total for _, v in top_words]
    return vec[:300] + [0.0] * (300 - len(vec))
```

Listing 13: providers.py — Keyword embedding generation

```
def cosine_similarity(a: list, b: list) -> float:
    dot   = sum(x * y for x, y in zip(a, b))
    mag_a = math.sqrt(sum(x * x for x in a))
    mag_b = math.sqrt(sum(x * x for x in b))
    return 0.0 if mag_a == 0 or mag_b == 0 else dot / (mag_a * mag_b)

def find_similar_analysis(new_embedding, past_embeddings):
    best_score, best_match = 0.0, None
```

```
    for entry in past_embeddings:
        score = cosine_similarity(new_embedding, entry["embedding"])
        if score > best_score:
            best_score, best_match = score, entry
    return best_match if best_match and best_score > 0.75 else None
```

Listing 14: providers.py — Cosine similarity and RAG retrieval

The historical context is then injected via `HISTORICAL_PLACEHOLDER` in the unified prompt:

```
rag_embedding = generate_embedding(contract_text[:8000])
similar = find_similar_analysis(rag_embedding, past_embeddings)
if similar:
    historical_context = (
        f"=== HISTORICAL CONTEXT ===\n"
        f"A similar {similar['contract_type']} contract was analyzed
            before "
        f"(file: {similar['filename']}, score: {similar['health_score
            ']}/100). "
        f"Compare this contract against that precedent."
    )
```

Listing 15: gemini_service.py — RAG context injection

9.4 Milestone 4: Persistence and Export

Activity 4.1 — SQLite Schema with SQLModel

Four tables are defined in `db.py` using SQLModel. The engine is a singleton with `check_same_thread=False` for Flask's multi-threaded request handling. Tables are created idempotently on every startup:

```
class AnalysisModel(SQLModel, table=True):
    __tablename__ = "analyses"
    id:            Optional[int] = Field(default=None, primary_key=True)
    session_id:    str  = Field(foreign_key="sessions.session_id", index
        =True)
    filename:      str  = Field(max_length=255)
    contract_type: str  = Field(default="UNKNOWN")
    full_text:     str  = Field(default="")
    analysis_json: str  = Field(default="{}")   # serialised JSON
    health_score:  int  = Field(default=0)
    embedding:     str  = Field(default="")     # JSON-encoded float
        list
    created_at:    datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc), index=True)

class ChatHistoryModel(SQLModel, table=True):
    __tablename__ = "chat_history"
    id:          Optional[int] = Field(default=None, primary_key=True)
    session_id:  str  = Field(foreign_key="sessions.session_id", index=
        True)
```

```
    analysis_id: Optional[int] = Field(default=None, foreign_key="
        analyses.id")
    role:        str  = Field(max_length=16)   # "user" | "assistant"
    content:     str  = Field(default="")
    timestamp:   datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc))
```

Listing 16: db.py — SQLModel table definitions

An analysis and its obligations are persisted atomically in a single `db.commit()` call:

```
def save_analysis(db, session_id, filename, contract_type,
                  full_text, analysis_json, health_score, embedding=None
                      ):
    get_or_create_session(db, session_id)
    analysis = AnalysisModel(
        session_id    = session_id,
        filename      = filename,
        contract_type = contract_type,
        full_text     = full_text[:50000],
        analysis_json = json.dumps(analysis_json),
        health_score  = health_score,
        embedding     = json.dumps(embedding or []),
    )
    db.add(analysis)
    db.commit()
    db.refresh(analysis)

    # Persist each obligation as its own row
    for obl in analysis_json.get("obligations", []):
        db.add(ObligationModel(
            analysis_id          = analysis.id,
            obligation           = obl.get("obligation", ""),
            deadline_description = obl.get("deadline_description", ""),
            party                = obl.get("party", ""),
            section              = obl.get("section", ""),
        ))
    db.commit()
    return analysis
```

Listing 17: db.py — Saving an analysis with embedding

Activity 4.2 — Session Memory (Memento Pattern)

`memory_service.py` implements the Memento pattern. Each `SessionMemory` instance stores the last 10 conversation turns (auto-trimmed), the active contract text, and a history of analyses. The `MemoryManager` acts as the Caretaker: it holds up to 500 sessions in an `OrderedDict` with LRU eviction:

```
class SessionMemory:
    def add_turn(self, role, content):
        self.turns.append({
            "role": role, "content": content,
```

```
            "timestamp": datetime.now(timezone.utc).isoformat(),
        })
        # Auto-trim to MAX_MEMORY_TURNS (10)
        if len(self.turns) > Config.MAX_MEMORY_TURNS:
            self.turns = self.turns[-Config.MAX_MEMORY_TURNS:]

    # Memento: save full state snapshot
    def save_snapshot(self):
        return {
            "turns":            copy.deepcopy(self.turns),
            "contract_text":    self.contract_text,
            "contract_name":    self.contract_name,
            "analysis_history": copy.deepcopy(self.analysis_history),
            "timestamp":        datetime.now(timezone.utc).isoformat(),
        }

    # Memento: restore from snapshot
    def restore_snapshot(self, snapshot):
        self.turns            = copy.deepcopy(snapshot.get("turns", []))
        self.contract_text    = snapshot.get("contract_text", "")
        self.contract_name    = snapshot.get("contract_name", "")
        self.analysis_history = copy.deepcopy(snapshot.get("
            analysis_history", []))

class MemoryManager:   # Caretaker
    def __init__(self):
        self._sessions: OrderedDict[str, SessionMemory] = OrderedDict()

    def get_session(self, session_id: str) -> SessionMemory:
        if session_id in self._sessions:
            self._sessions.move_to_end(session_id)   # LRU bump
            return self._sessions[session_id]
        mem = SessionMemory()
        self._sessions[session_id] = mem
        if len(self._sessions) > _MAX_SESSIONS:       # evict oldest
            self._sessions.popitem(last=False)
        return mem
```

Listing 18: memory_service.py — SessionMemory with Memento pattern

Activity 4.3 — PDF Export Generation

The `/api/export/<id>` route retrieves a stored analysis from SQLite, passes it to `export_service.py`, which uses FPDF2 to render a multi-page PDF report. The file is streamed back as a binary download with a descriptive filename:

```
@app.route("/api/export/<int:analysis_id>", methods=["GET"])
def api_export(analysis_id):
    row = g.db.exec(select(AnalysisModel).where(
        AnalysisModel.id == analysis_id,
        AnalysisModel.session_id == _sid()
    )).first()
    if not row:
        abort(404)
```

```
    pdf_bytes = generate_pdf(row)   # FPDF2 rendering in export_service.
        py
    download_name = f"clauseclear_{row.filename}_{row.id}.pdf"

    return send_file(
        io.BytesIO(pdf_bytes),
        mimetype="application/pdf",
        as_attachment=True,
        download_name=download_name,
    )
```

Listing 19: app.py — PDF export route

10 Results & Verification

System Output Metrics

- Unified analysis completion time: 12–22 seconds (avg. ∼16 seconds)
- PDF export generation time: < 0.5 seconds
- JSON parse success rate: ∼98% (ensured by structured outputs + self-critique)
- RAG context injection: successfully injects historical context on repeated documents
- Max contract size tested: 14,800 characters (within 15,000 char limit)
- Supported input formats: PDF, DOCX, TXT, PNG, JPG (max 10 MB)
- Session memory retention: 10 turns per session (Memento Pattern)

Screenshots

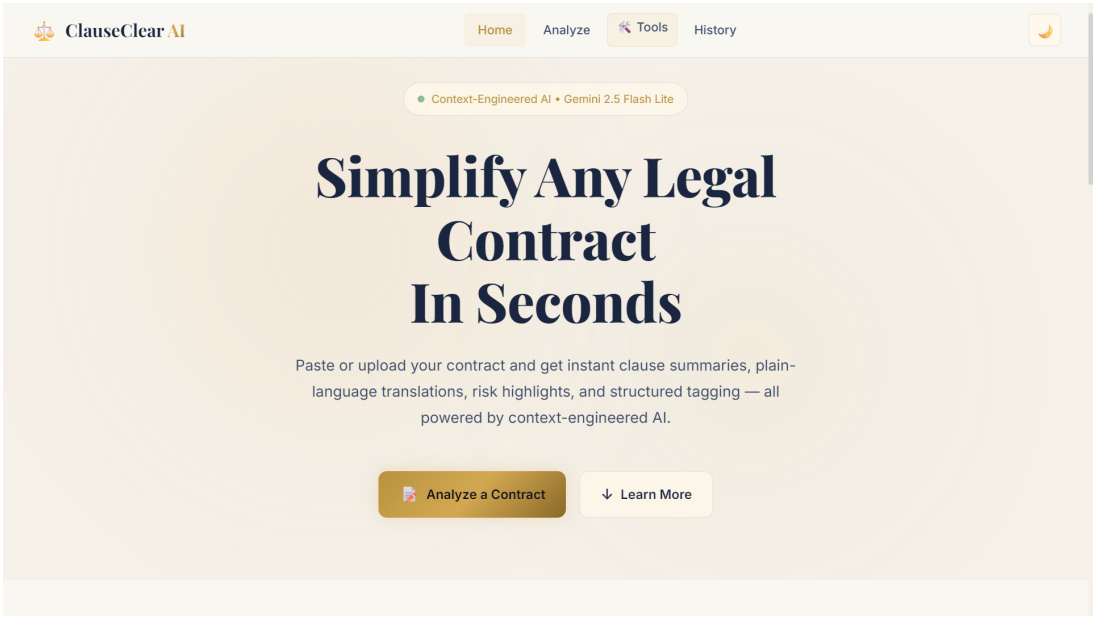

Figure 4: Application Landing Page

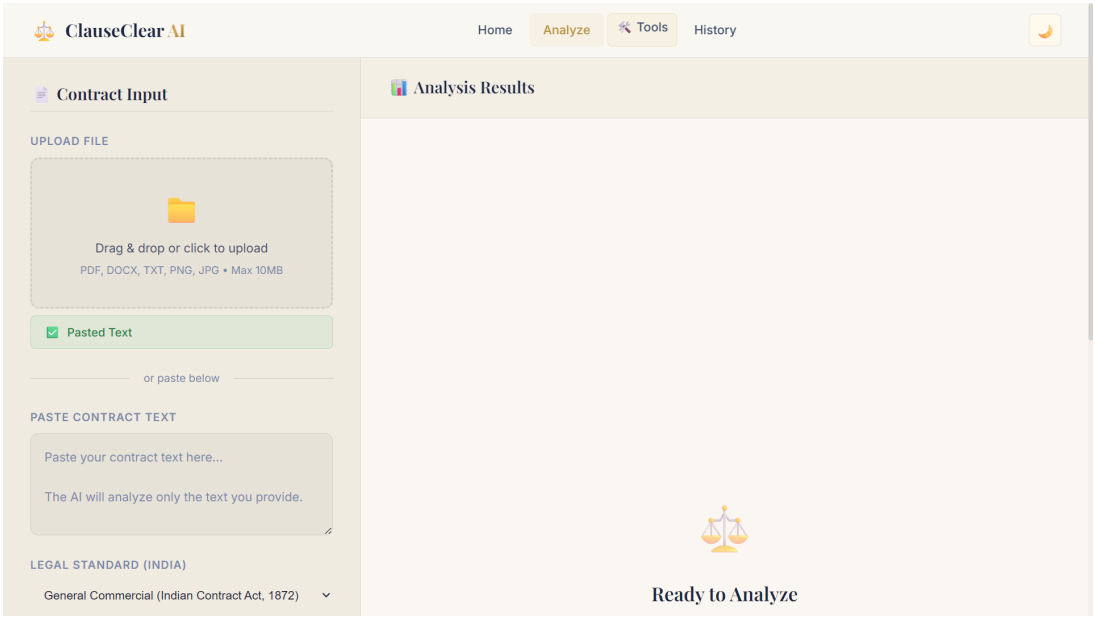

Figure 5: Contract Analysis Dashboard

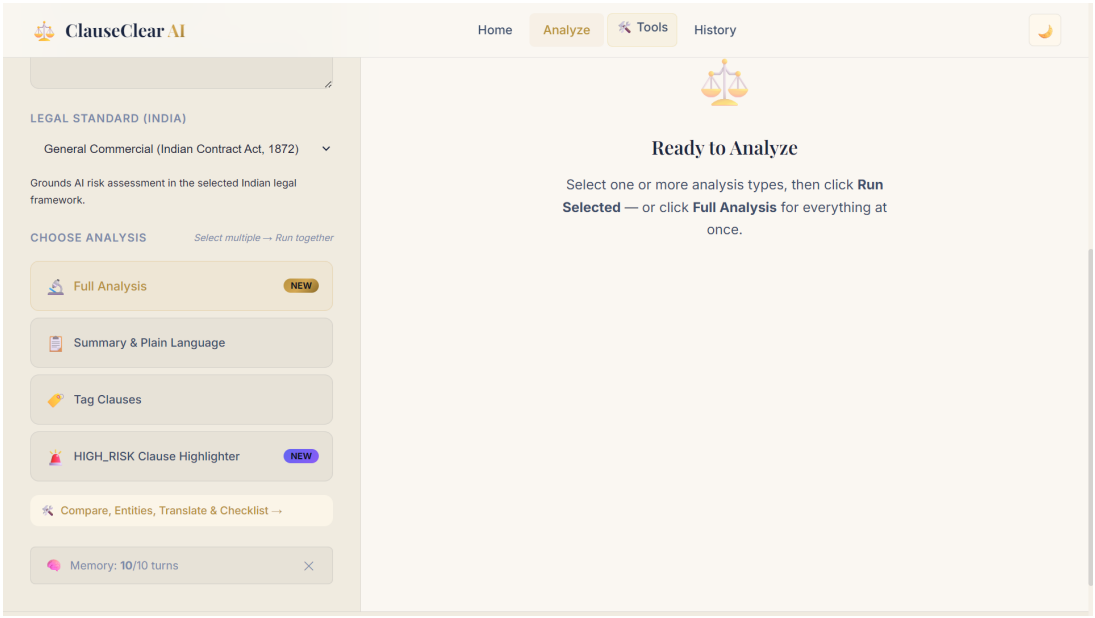

Figure 6: Analysis Type Selection with Memory Indicator

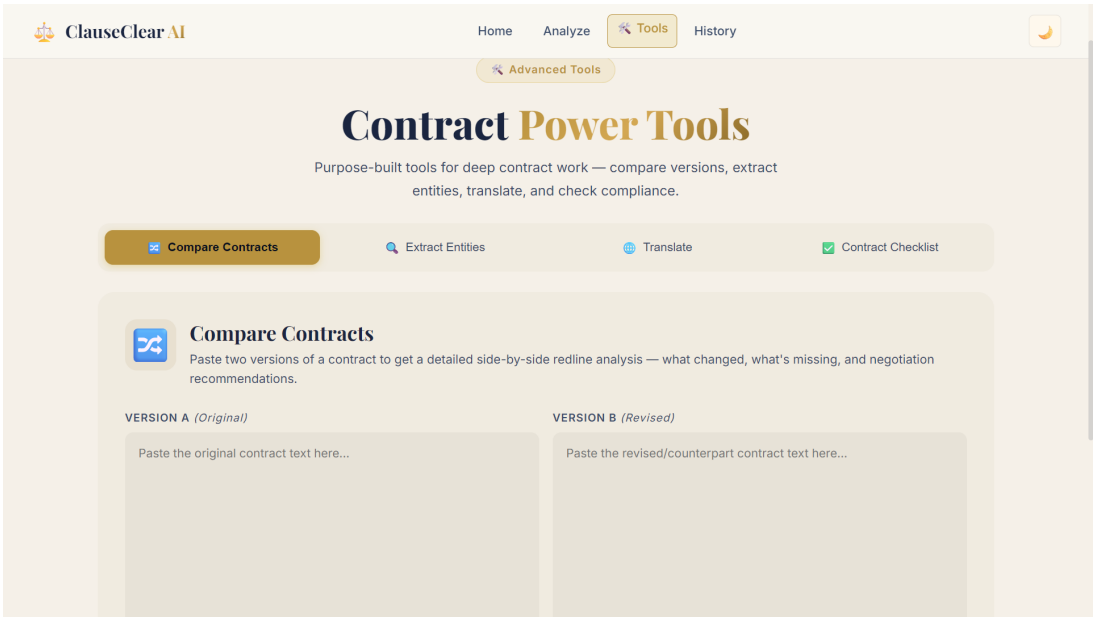

Figure 7: Contract Power Tools

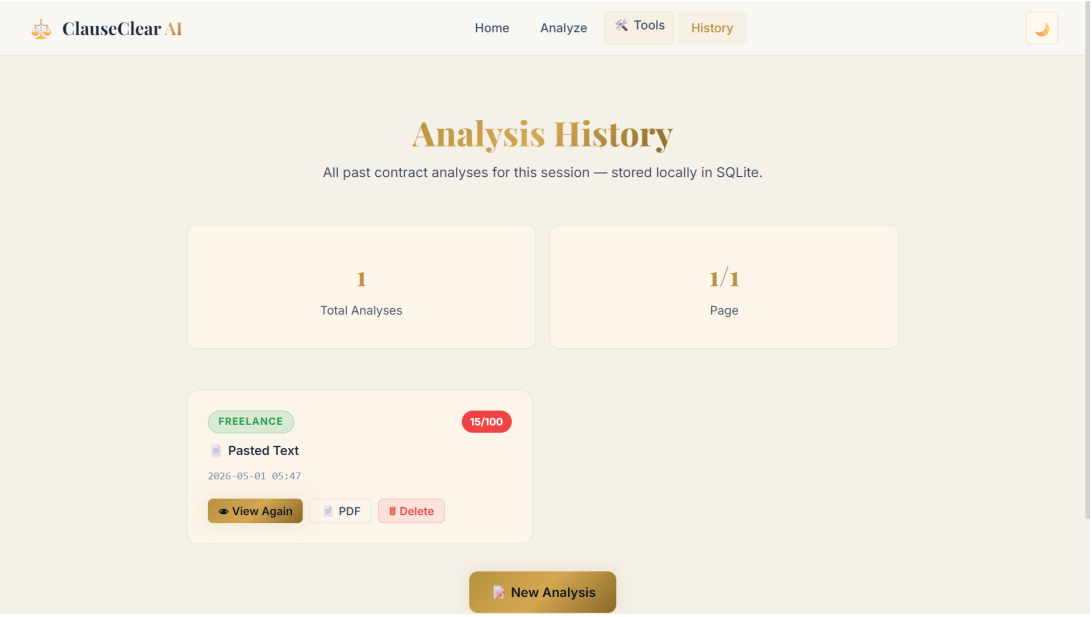

Figure 8: Analysis History with PDF Export

11 Advantages & Limitations

Advantages

- **Legally Grounded Assessment:** Ties `HIGH_RISK` flags to specific Indian statutes, making output directly actionable.
- **Production Context Engineering:** Six simultaneous principles with a self-critique loop demonstrate enterprise-level design.
- **Self-Contained RAG:** Uses pure Python cosine similarity and SQLite, requiring no external vector databases.
- **Four-Provider AI Cascade:** Gemini → DeepSeek → OpenAI → Claude ensures high availability even during individual API outages.
- **Zero Frontend Dependency:** No React, Vue, or Angular — pure Vanilla JS with professional glassmorphism UI.

Limitations

- **API Dependency:** Relies on external AI APIs; all four providers being down simultaneously would make the system unavailable.
- **Character Limit:** Contracts over 15,000 characters must be chunked

or truncated.

- **Hallucination Risk:** Despite multi-layer constraints, the AI may occasionally misclassify complex clauses.
- **Local Storage:** SQLite database is local; cross-device access requires cloud migration.

12 Future Enhancements

Short-Term

- Implement semantic chunking for documents exceeding 15,000 characters.
- Add real-time contract redlining — highlight risky clauses inline in the uploaded document.

Medium-Term

- Add user authentication (OAuth) for cross-device session persistence.
- Build a negotiation assistant mode — AI suggests statute-compliant alternative clause language.

Long-Term

- Multi-jurisdiction playbooks (UAE, Singapore, UK).
- React Native mobile application for on-the-go contract review.

13 Conclusion

ClauseClear AI demonstrates how Generative AI can be applied to legal contract analysis while maintaining rigorous output quality through systematic context engineering. Grounding risk assessments in the Contract Act 1872, DPDP Act 2023, and IT Act 2000 creates a legally contextual tool for Indian jurisdictions.

The platform combines a secure REST API, persistent storage with keyword embeddings, session memory, and a zero-dependency frontend in a single robust Flask application. The six-principle context engineering pipeline — augmented by a four-provider AI cascade and an autonomous self-critique

loop — makes ClauseClear AI a production-grade reference implementation for applying Generative AI to domain-specific professional problems.

14 Appendix

GitHub Repository

GitHub: https://github.com/THARUN9959/Clause-Clear-AI

Project Structure

- `app.py` — Core Flask application: 13 API routes, coordinates all service calls, initializes SQLite.
- `config.py` — Loads environment variables and global constants; raises on missing required keys.
- `db.py` — SQLModel schema definitions (`AnalysisModel`, `ChatHistoryModel`, `ObligationModel`) and CRUD helpers.
- `services/gemini_service.py` — Public AI service API: orchestrates classify → embed → analyze → self-critique pipeline.
- `services/providers.py` — Four-provider AI dispatcher: Gemini, DeepSeek, OpenAI, Claude. Embedding and cosine similarity utilities.
- `services/memory_service.py` — Memento Pattern session memory: `SessionMemory` + LRU `MemoryManager`.
- `services/playbooks.py` — Indian Legal Playbook strings mapped to UI dropdown values.
- `services/prompts.py` — All prompt templates, JSON schema definitions, and feature prompt strings.
- `services/benchmark_service.py` — Market-standard clause range data for 7 contract types.
- `services/export_service.py` — FPDF2-based PDF report generator.
- `templates/` — Jinja2 HTML templates (`index.html`, `analyze.html`, `tools.html`, `history.html`).

- `static/` — CSS (glassmorphism variables) and JS (async fetch handlers, DOM rendering).

API Endpoints

- `POST /api/upload` — Multipart file upload (PDF/DOCX/Image). Returns text preview.
- `POST /api/analyze` — Body: `{contract_text, evaluation_standard}`. Returns full analysis JSON.
- `POST /api/analyze/feature` — Body: `{feature, contract_text}`. Returns feature-specific result.
- `POST /api/chat` — Body: `{message, analysis_id}`. Returns `{answer, relevant_clauses, confidence}`.
- `GET /api/history/<id>` — Returns stored analysis JSON for a past session.
- `DELETE /api/history/<id>` — Cascade-deletes analysis and all linked chat/obligation rows.
- `GET /api/export/<id>` — Streams PDF binary download.
- `GET /api/memory` — Returns current session memory state summary.
- `POST /api/clear-memory` — Clears in-memory session turns.

Running the Application

```
# 1. Clone and enter the project
git clone https://github.com/THARUN9959/Clause-Clear-AI.git
cd GENAI-2

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate        # Windows

# 3. Install dependencies
pip install flask google-genai sqlmodel fpdf2 PyPDF2 python-docx \
    python-dotenv anthropic flask-wtf flask-limiter

# 4. Create .env file
GEMINI_API_KEY=your_google_ai_studio_key
ANTHROPIC_API_KEY=your_anthropic_key
FLASK_SECRET_KEY=your_32_byte_hex_string
FLASK_DEBUG=true

```

```
# 5. Start the application
python app.py
# Visit: http://localhost:5000
```

Listing 20: Full setup and run instructions